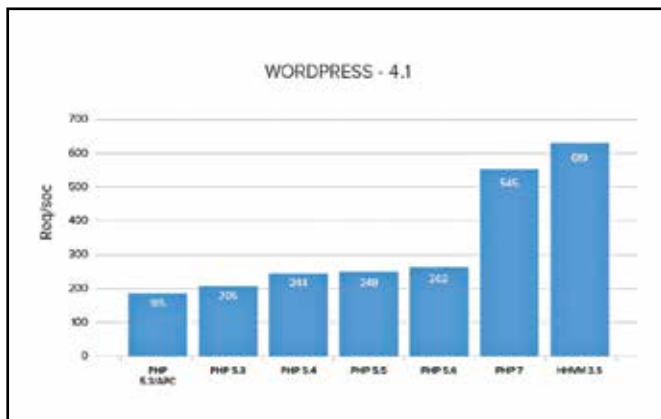


But why PHP 7, not PHP 6? Well, things didn't turn out very good with Unicode. The PHP fraternity previously had goals to release a new major version back in 2005. The new version was supposed to implement Unicode and a whole lot of other features. Most of those other new features were added in the subsequent PHP version 5.3 and later, except the Unicode. In the meanwhile, PHP 6 always existed as an experimental project and a lot was talked and



Performance improvement with PHP7

published about it, but it could never make it up to the production phase. Thus, considering the name as PHP 7 was always a viable option to avoid confusion and hindrance to the mass regarding the contents and features of the newest version.

A lot being already discussed about the basic advantages and naming pattern, it's time we look into the new features and functionalities that PHP 7 brings along with it.

1. New Operators

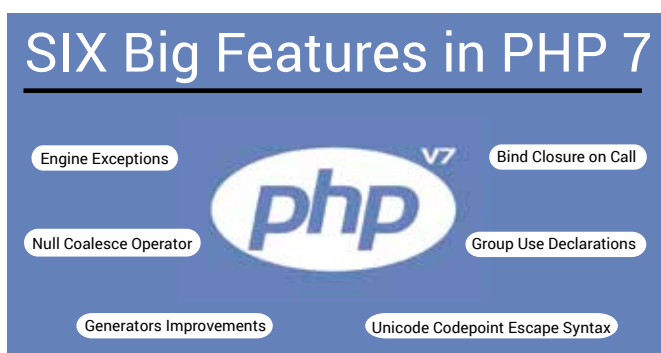
Finally, PHP brings in two new operators and these operators are definitely going to make your life much easier.

The Spaceship Operator (<=>)

This operator goes by the official name of Combined Comparison Operator. The notation of this operator looks kind of like a spaceship (if you have ever seen one). The spaceship operator is built up by combining three different operators - less than, equal, and greater than. Although, these operators are combined together, each of these operators is checked individually from left to right.

```
left <=> right
```

Consider the above line of PHP code which uses the spaceship operator -



- If the left value is less than the value the right value, the operator will return -1.
- If the left value is equal to the value the right value, the operator will return 0.
- If the left value is greater than the value the right value, the operator will return 1.

Note: If you were to do the same in PHP version 5.6 or below, we would have to write the following piece of code -

```
left < right? return -1
left = right? return 0
left > right? return 1
```

Null Coalesce Operator (??)

If you have even tried learning PHP, there is no way you wouldn't have come across the **isset()** function and later you might have also realized that using this function repeatedly to check if a value is set or null is way too old school. The Null Coalesce Operator brings an alternative to **isset()** function. It provides us with a shorthand alternate to what **isset()** does with inline comparison.

```
$source = $defined ?? "Undefined";
```

Consider the above line of PHP code which uses the null coalesce operator. The above line of code first checks if the variable defined is set or null. If **defined** is set, it will assign its value to the variable source. If variable **defined** is not set i.e. **null**, it will assign "**Undefined**" to the variable source.

Note: If you were to do the same in PHP version 5.6 or below, we would have to write the following piece of code -

```
if (empty($defined))
    $source = $defined;
else
    $source = "Undefined";
```

Shouldn't you be thanking PHP 7, already?

2. Awesome Error Handling

In PHP 5.6 or below, error handling was such a pain. In case there was a fatal error, the framework wouldn't even invoke the error handler and consequently stop the script from processing. Thus, the white screen of incredibility. Well, gone are the bad days. In PHP 7, if there is a recoverable or fatal error, an exception will be thrown. Yes, no more white screens. Fatal errors also include errors such as running out of memory and an uncaught exception.

In PHP 7, Errors and Exceptions work pretty much in a similar way. Both implement the **Throwable** class. Yes, that means that now you can use **Throwable** in try/catch blocks for both Errors and Exceptions. However, in PHP 7 for Error, we now have some more specific error - **AssertionError**, **ArithmeticError**, **DivisionByZeroError**, **TypeError** and **ParseError**.

Consider the line of PHP code below as an example which uses **Throwable** to handle Errors and Exceptions -

```
//Error as Throwable
try {
    someFunction();
} catch (Throwable $t) {
    echo "Throwable: ".$t->getMessage().PHP_EOL;
}
```